

Sobel Filter on FPGA

Authors:

WhisperingRock

Professor:

Class:

Quarter:

September 1, 2026

Contents

Contents	1
1 Theory of the Sobel Operator	2
1.1 Grayscale Operator	2
1.2 Sobel Operator	2
2 System Architecture	3
3 FPGA IP	4
3.1 Line Buffer	4
4 Results	6
5 Analysis	7
6 Conclusion	8
7 Appendix	9
7.1 SystemVerilog	9
7.1.1 Line Buffer	9
8 References	13

Chapter 1

Theory of the Sobel Operator

1.1 Grayscale Operator

This implementation requires that the RGB image (TODO img type) be flattened into a single intensity value (a grayscale value). This can be done with a few methods.

- Average :

$$p = \frac{R + G + B}{3}$$

- ITU-R B.709 Luminance

$$p = 0.2126R + 0.7152G + 0.0722B$$

I opt'd to use Luminance for its accepted standard.

1.2 Sobel Operator

Uses two 3x3 kernels which are convolved with the original (grayscale) image to calculate approximations of the derivatives. Each kernel, one for horizontal change and one for vertical change, performs a low pass Gaussian filtering [1 2 1] as well a discrete differentiation [+1 0 -1] on each pixel of the image.

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$
$$\mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Consider the resulting components :

- Magnitude :

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

- Direction of gradient :

$$\Theta = \arctan \left(\frac{\mathbf{G}_y}{\mathbf{G}_x} \right)$$

The results of the operator produce a cheap edge detector that performs some smoothing while retaining a DC gain of 1. For the scope of this project, I'm only using the magnitude.

Chapter 2

System Architecture

Chapter 3

FPGA IP

3.1 Line Buffer

The LineBuffer is designed to ...

- incrementally collect individual input pixel values into the internal storage, using *DIN_AVAIL* and *DIN*, on each clock cycle. *DIN_AVAIL* enables writes while also moving the write pointer to the next position.
- always output a 3 pack of neighbor pixels to *DOUT*, only updating and shifting on a clock'd *DOUT_AVAIL*.
- naturally rollover internal readPtr and writePtr to 512 count (pixels) or reset to zero using *RST*. One might consider the buffer circular using the rollover.

Notes:

- Need at least three writes at or ahead of readPtr before using output data to avoid reading jump.

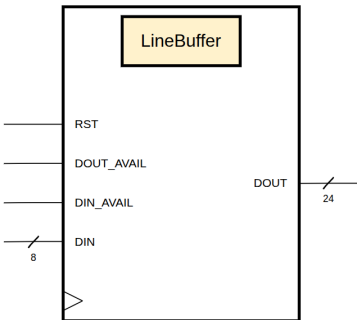


Figure 3.1: LineBuffer Block

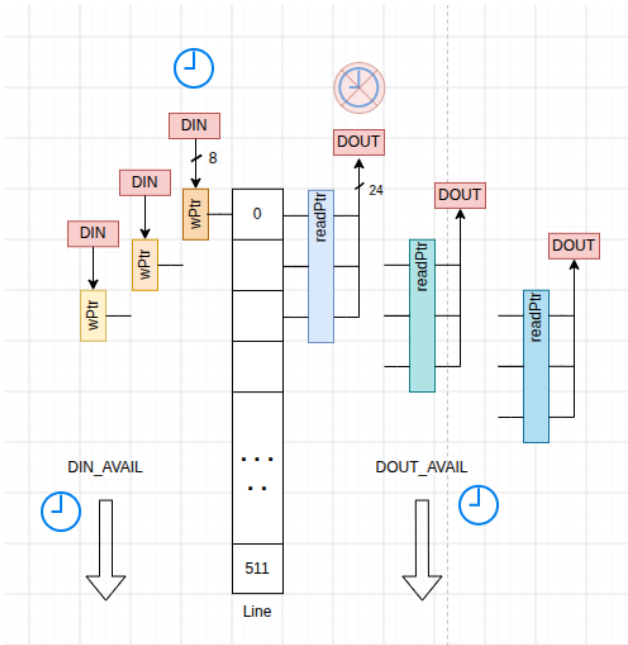


Figure 3.2: LineBuffer Visualized

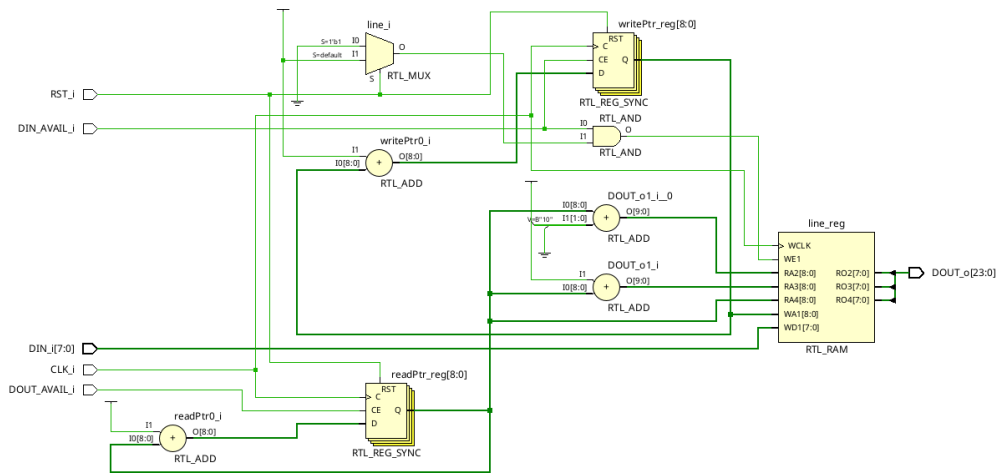


Figure 3.3: LineBuffer RTL

Code Snippets :

- [LineBuffer.sv 7.1](#)
- [LineBuffer_tb.sv 7.2](#)

Chapter 4

Results

Chapter 5

Analysis

Chapter 6

Conclusion

Chapter 7

Appendix

7.1 SystemVerilog

7.1.1 Line Buffer

Listing 7.1: LineBuffer.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 09:16:29 AM
7  // Design Name:
8  // Module Name: LineBuffer
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:  Gathers a single row (512) of pixel data, one pixel at a time,
13 //                and outputs a single row (3) for a 3x3 kernel. The input_avail
14 //                signal indicates the next pixel is ready to be read and the
15 //                output_avail indicates the downstream module is ready for a
16 //                kernel window slide.
17 //
18 //                How to use :
19 //                - Fill the row largely unrestricted but prevent rollover until ready for new
20 //                row
21 //                - Only slide kernel row when sobel operation has finished that that position
22 //                - reset can also be used rather than rollover to start a new row/window
23 // Dependencies:
24 //
25 // Revision:
26 // Revision 0.01 - File Created
27 // Additional Comments:
28 //
29 ///////////////////////////////////////////////////////////////////
30
31
32 module LineBuffer
33 #(parameter int unsigned PWIDTH = 512,
34   int unsigned PHEIGHT = 512)
35 (
36   input logic      CLK_i,
37   input logic      RST_i,
38   input logic [7:0] DIN_i,
39   input logic      DIN_AVAIL_i, // input data ready to be read/sent to this mod
40   input logic      DOUT_AVAIL_i, // output data ready to be read/recieved from this mod
41   output logic [23:0] DOUT_o // packing 3 byte/pixels into a single word. Not using
42     packed array to reduce dimension
43 );
44
45 // ~~~~ local params/alloc ~~~~
46 logic [7:0] line [0:PWIDTH-1]; // Single line contains 512 bytes worth of pixels
47 logic [8:0] writePtr; // 9 bits covers range of PIXELHEIGHT s.t. 2^9 = 512
48 logic [8:0] readPtr;
49
50 // ~~~~ sync logic ~~~~
51 always_ff @(posedge CLK_i)
52 begin
53
54   // ~~ priority 0 : reset r/w ptrs to zero place ~~
55   if(RST_i) begin
56     writePtr <= 0;
57     readPtr <= 0;
58   end
59
60   // ~~ priority 1 : read/write ~~
61   else begin
62
63     // ~ Store pixel i in row/line when available ~
```

```

64     if(DIN_AVAIL_i == 1'b1) begin
65         line[writePtr] <= DIN_i;
66         writePtr      <= writePtr + 1'b1;
67     end
68
69     // ~ Slide the kernel window (this row in particular) right 1 when prompted ~
70     if(DOUT_AVAIL_i == 1'b1) begin
71         readPtr      <= readPtr + 1'b1;
72     end
73 end
74
75 end
76
77 // ~~~~ comb logic ~~~~
78 assign DOUT_o = {line[readPtr], line[readPtr+1], line[readPtr+2]}; // pack pixels, for
79                             one row in a 3x3 kernel, into a single output
80
81 endmodule

```

Listing 7.2: LineBuffer_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/31/2026 10:33:22 AM
7  // Design Name:
8  // Module Name: LineBuffer_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module LineBuffer_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local param/allocs ~~~~
29     // ~~ ports ~~
30     logic      clk;
31     logic      reset;
32     logic [7:0] din;
33     logic      din_avail;
34     logic      dout_avail;
35     logic [23:0] dout;
36     // ~~ consts ~~
37     localparam int unsigned PWIDTH  = 12; // smaller for ease of testing
38     localparam int unsigned PHEIGHT = 12;
39     // ~~ testing ~~
40     testcase tc;
41     logic [31:0] tnum;
42     bit [11:0][7:0] arr;
43
44
45     // ~~~~ module instances ~~~~
46     LineBuffer #(PIXELWIDTH(PWIDTH), .PIXELHEIGHT(PHEIGHT)) uut
47     (
48         .CLK_i(clk),          // 1'b
49         .RST_i(reset),        // 1'b
50         .DIN_i(din),          // 8'b
51         .DIN_AVAIL_i(din_avail), // 1'b
52         .DOUT_AVAIL_i(dout_avail), // 1'b
53         .DOUT_o(dout)          // 24'b
54     );
55
56     // ~~~~ heartbeat (10ns) ~~~~
57     always begin
58         #5;
59         clk <= !clk;
60     end
61
62     // ~~~~ testing ~~~~
63     initial begin
64
65         // ~~ class instances ~~
66         tc      = new();
67
68         // ~~ defaults ~~
69         clk      = 1'b0;
70         reset    = 1'b1;

```

```

71     din          = 8'h00;
72     din_avail    = 1'b0;
73     dout_avail   = 1'b0;
74     #100;
75     reset        = 1'b0;
76     #5;
77
78
79     // ~~ TC1 ~~
80     tc.new_test("Filling Line");
81     tnum         = tc.get_testnum();
82
83     arr          = 96'h12_11_10_09_08_07_06_05_04_03_02_01;
84     din_avail    = 1'b1;
85
86     tc.print_subtest("2/3 of writes");
87     din          = arr[0];
88     #5;
89     assert(dout === {arr[0], 16'hXX_XX})      else tc.err("DOUT error");
90     #5;
91
92     din          = arr[1];
93     #5;
94     assert(dout === {arr[0], arr[1], 8'hXX})   else tc.err("DOUT error");
95     #5;
96
97     tc.print_subtest("Disable write");
98     din_avail    = 1'b0;
99     din          = arr[2];
100    #5;
101    assert(dout === {arr[0], arr[1], 8'hXX})     else tc.err("DOUT error");
102    #5;
103
104    tc.print_subtest("Re-enable write for 3/3 writes");
105    din_avail    = 1'b1;
106    #5;
107    assert(dout === {arr[0], arr[1], arr[2]})     else tc.err("DOUT error");
108    #5;
109
110    tc.print_subtest("Full buffer prevents new data");
111    din          = arr[3];
112    #5;
113    assert(dout === {arr[0], arr[1], arr[2]})     else tc.err("DOUT error");
114    #5;
115    tc.test_done();
116
117
118    // ~~ TC2 ~~
119    tc.new_test("Reset ptrs");
120    tnum         = tc.get_testnum();
121    reset        = 1'b1;
122    din_avail    = 1'b0;
123    dout_avail   = 1'b0;
124    #10;
125
126    // overwrite first 3 spots with zeros
127    reset        = 1'b0;
128    din_avail    = 1'b1;
129    din          = 8'h00;
130    #30;
131    assert(dout === {24'h00_00_00})      else tc.err("DOUT error");
132    tc.test_done();
133
134    // ~~ TC3 ~~
135    tc.new_test("Fill to top, then empty");
136    tnum         = tc.get_testnum();
137    // place writePtr back to zero
138    reset        = 1'b1;
139    #10;
140    reset        = 1'b0;
141    din_avail    = 1'b1;
142    dout_avail   = 1'b0;
143
144    // fill
145    din          = arr[0];
146    #10;
147    din          = arr[1];
148    #10;
149    din          = arr[2];
150    #10;
151
152    // fill
153    for(int i = 3; i < 12; i++) begin
154        din      = arr[i];
155        #5;
156        assert(dout === {arr[0], arr[1], arr[2]}) else tc.err("DOUT error");
157        #5;
158    end
159
160    // empty
161    din_avail    = 1'b0;

```

```

162     dout_avail      = 1'b1;
163     for(int i = 3; i < 12; i++) begin
164         #5;
165         assert(dout == {arr[i-2], arr[i-1], arr[i]}) else tc.err("DOUT error");
166         #5;
167     end
168
169     #5;
170     assert(dout == {arr[10], arr[11], 8'hXX})      else tc.err("DOUT error");
171     #10;
172     assert(dout == {arr[11], 16'hXX_XX})           else tc.err("DOUT error");
173     #10;
174     assert(dout == {24'hXX_XX_XX})                 else tc.err("DOUT error");
175     #5;
176
177     tc.test_done();
178
179
180     // ~~ TC4 ~~
181     tc.new_test("R/W synch'd rollover");
182     tnum      = tc.get_testnum();
183
184     //todo
185
186
187     tc.test_done();
188
189
190
191 end
192
193
194 endmodule

```

Chapter 8

References

Youtube Inspiration